CUSTOMER RELATIONSHIP MANAGEMENT WITH DEVSECOPS

21IPE315P - Cloud Product and Platform Engineering Report

Submitted by

ANANT SHRIVASTAVA (RA2311028010116)

SHREYA TIWARI (RA2311028010141)

ARHAM RASHEED (RA2311028010120)

Under the Guidance of

Dr. Sivakumar M.

Assistant Professor, Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE and ENGINEERING

with specialization in CLOUD COMPUTING



DEPARTMENT OF NETWORKING AND COMMUNICATIONS

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2026

Department of Networking and Communications
SRM Institute of Science and Technology
Own Work Declaration Form

Degree/ Course	**:**	B. Tech in Computer Science and Engineering with specialization in Cloud Computing
Student Name	**:**	Anant Shrivastava, Shreya Tiwari, Arham Rasheed
Student Name	**:**	RA2311028010116, RA2311028010141, RA2311028010120
Title of Work	**:**	CUSTOMER RELATIONSHIP MANAGEMENT w DevSecOps

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

We are aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

Anant Shrivastava (RA2311028010116)	Shreya Tiwari (RA2311028010141)	Arham Rasheed (RA2311028010120)

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

BONAFIDE CERTIFICATE

Certified that 21IPE333P – Devops Project report titled "**CUSTOMER RELATIONSHIP MANAGEMENT WITH DEVSECOPS"** is the Bonafide work of **"ANANT SHRIVASTAVA [RA2311028010116], SHREYA TIWARI [RA2311028010141], ARHAM RASHEED [RA2311028010120]**" who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. Sivakumar M.
Assistant Professor
Department of Networking and Communications

Dr. M. Lakshmi.
HOD & Head
Department of Networking and Communications

ACKNOWLEDGE

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr.C. Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. M. Lakshmi**, Professor & Head, Department of Networking and Communications, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We register our immeasurable thanks to our Faculty Advisor, **Dr. B Lakshmi Dhevi**, Department of Networking and Communications, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our Faculty, **Dr. Sivakumar M.**, Department of Networking and Communications, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Networking and Communications, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Anant Shrivastava (RA2311028010116)

Shreya Tiwari (RA2311028010141)

Arham Rasheed (RA2311028010120)

ABSTRACT

Cloud infrastructure management plays a crucial role in enabling organizations to deploy and manage applications efficiently; however, it often creates challenges when handling complex environments without automation. Traditional infrastructure management methods rely heavily on manual provisioning and configuration, which may not always be consistent and can lead to delays, errors, or system failures. With the advancement of cloud computing and DevOps practices, automated infrastructure provisioning systems have emerged as a promising solution to address these operational challenges. This project focuses on designing a complete Retail CRM system for customer relationship management, built around the online_retail_listing.csv dataset. Multiple technologies have been integrated to develop an efficient and reliable system.

FastAPI is used as the core backend framework to define API endpoints, while React/Vite is employed for the frontend dashboard to provide a user-friendly interface. MLFlow is utilized to track the machine learning pipeline, enabling the systematic training and versioning of customer value prediction models. Jenkins is integrated to orchestrate deployment pipelines, enabling the system to execute infrastructure workflows in a controlled and automated manner.

To enhance usability, the deployed CRM is monitored and managed through a centralized dashboard interface, allowing users to track customer lists, view detailed analytics, and trigger on-demand predictions. Monitoring tools are implicitly supported by the API endpoints (e.g., /health), while MLFlow provides deep insights into model performance (tracked in backend/ml/mlruns).

The system is tested under different operational conditions—such as running tests via pytest or training the model (python ml/train.py)—achieving a high success rate in data processing and prediction tasks, demonstrating its effectiveness in real-time CRM automation.

By combining Backend API functionality (FastAPI), ML Model Training (scikit-learn + MLFlow), Frontend Visualization (React), and CI/CD Orchestration (Jenkinsfile) into a unified workflow, the system provides a comprehensive solution for DevOps-based Retail CRM management. Although performance may vary based on data complexity or cloud conditions, the system shows strong potential for scalability. With further improvements, it can evolve into a reliable solution for enterprise-level customer relationship management and analytics.

TABLE OF CONTENT

ABSTRACT		**v**
TABLE OF CONTENT		**vi**
ABBREVATIONS		**x**
CHAPTER NO.	**TITLE**	**PAGE NO.**
1	**INTRODUCTION**	**1**
	1.1 Introduction to Project	1
	1.2 Problem Statement and Description	2
	1.3 Research Objectives	2
	1.3.1 Challenges in Modern Deployment Systems	3
	1.3.2 Research Gap	4
2	**TECHNOLOGY STACK**	**5**
	2.1 Backend Framework: FastAPI	5
	2.2 Data Processing & Analytics: Python / Pandas	6
	2.3 Machine Learning: Scikit-learn	6
	2.4 ML Model Tracking: MLFlow	6
	2.5 Frontend: React + Vite	7
	2.6 CI/CD Orchestration: Jenkins	8
	2.7 Deployment Artifacts: Joblib(customer_value_model.joblib)	8
3	**SYSTEM ARCHITECTURE**	**9**
	3.1 System Architecture	9
	3.2 Project Methodology	10
	3.3 DevOps Pipeline	11
	3.4 Implementation	12
4	**RESULTS AND DISCUSSIONS**	**13**
	4.1 Testing	13
	4.2 Results	13
	4.3 Challenges	15

5 CONCLUSION AND FUTURE ENHANCEMENT 16
5.1 Conclusion 16
5.2 Future Enhancements 17
REFERENCES 18
APPENDIX A IMPLEMENTATION 20
APPENDIX B OUTPUT 22

ABBREVIATIONS

- CI/CD → Continuous Integration / Continuous Deployment
- IaC → Infrastructure as Code
- AWS → Amazon Web Services
- EC2 → Elastic Compute Cloud
- VPC → Virtual Private Cloud
- SSH → Secure Shell
- API → Application Programming Interface
- JSON → JavaScript Object Notation
- YAML → Yet Another Markup Language
- HTTP → HyperText Transfer Protocol
- DB → Database
- UI → User Interface
- UX → User Experience
- VM → Virtual Machine
- OS → Operating System
- SSE → Server-Sent Events
- REST → Representational State Transfer
- CPU → Central Processing Unit
- RAM → Random Access Memory
- IP → Internet Protocol
- DNS → Domain Name System

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

The difficulty in managing application logic and data across different environments (e.g., local development vs. production) is one of the most significant challenges in modern software systems. This occurs due to the absence of a unified automation platform, which makes it difficult for organizations to provision, configure, and monitor customer analytics services efficiently. As a result, it may affect their ability to deploy features, maintain data reliability, and scale prediction capabilities effectively. There are different operational configurations, such as running locally via uvicorn vs. deploying on a cloud VM setup. The variation between these systems further increases operational complexity. Therefore, there is always a need for systems that can automate CRM management into a more standardized and controlled process.

Automation-based infrastructure provisioning techniques using Infrastructure as Code (IaC) tools have proven to be effective in managing the application environment. Configuration files provide a clear and repeatable way to define the system's structure (e.g., defining API routes or ML model dependencies), which makes them useful for deployment tasks. However, managing CRM workflows still requires skilled professionals who are capable of handling data parsing and model training efficiently. Due to the growing need for rapid feature delivery and continuous improvement, there is an increased dependency on such expertise. Apart from that, factors such as human error (e.g., incorrect date format input), time constraints, and lack of visibility into model performance can also interfere with effective system management. Consequently, a DevOps-based CRM system is needed to assist in automating and managing these tasks accurately.

With the rapid growth in the field of cloud computing and DevOps practices, automated infrastructure management has become more feasible and efficient than ever before. In particular, Infrastructure as Code tools, along with configuration management systems (like Python scripts), have the ability to define and deploy services programmatically. Technologies such as Terraform can provision the necessary hosting environment, while tools like Jenkins can orchestrate workflows that execute model training (train.py) reliably after deployment. These tools provide

efficient performance with reduced manual effort and support automated workflows for data processing and prediction serving.

The purpose of this project is to develop a system for automated CRM functionality provisioning and management using DevOps tools. The project approach consists of three main stages — data parsing/API definition (using FastAPI), ML model training (using scikit-learn + MLFlow), and execution management (using Jenkins pipelines) for controlled and observable deployment.

1.2 Problem Statement and Description

The conventional methods of managing CRM functionality mainly depend on manual data loading, ad-hoc API calls, and experienced developers. However, even though this approach is widely used, it also presents several limitations. Firstly, feature deployment (like adding a new customer filter) is not always efficient in real-time and may require prior setup and manual intervention. Besides, this method depends heavily on skilled professionals, making it less scalable. The involvement of human factors such as data cleaning errors (e.g., incorrect decimal comma usage), delays, and inconsistencies can lead to unreliable system deployments.

At the same time, with the increasing demand for scalable and reliable customer insights in areas such as retail sales analysis, enterprise applications, and data science platforms, the need for automated CRM management is growing rapidly. However, there are not enough standardized solutions to support seamless deployment across different environments (e.g., local vs. production). This makes it difficult for organizations to manage infrastructure efficiently in dynamic conditions, affecting system performance and service availability. The major challenges in CRM automation include:

- Variability in Data Configurations: Infrastructure setups may vary based on the dataset structure or required filters, making it difficult to maintain consistent data handling logic.
- ML Model Versioning Differences: Different training runs might produce slightly different models; managing which model is active becomes complex without tracking.
- Sensitivity to API/Data Conditions: Connectivity issues or poor data quality (e.g., non-positive quantities) can affect prediction reliability.

- Real-time Execution Constraints: The system must provision and train the ML model quickly to ensure smooth, up-to-date customer value predictions.

Such a system can reduce dependency on manual processes, improve deployment efficiency, and enable real-time infrastructure management, contributing to a more reliable and scalable computing environment for retail analytics.

1.3 Research Objectives

Based on the identified research gap, this project aims to design and implement a robust, secure, and scalable **Retail CRM DevOps framework**. The primary objectives are as follows:

- Architect a Declarative Data & API Layer:

  Design and provision a fully automated, declarative application environment using IaC principles. This includes defining the FastAPI service endpoints (e.g., /api/customers) via configuration, ensuring consistent data parsing rules (; delimiter + decimal comma support), thereby eliminating manual configuration drift and ensuring environment consistency.

- Implement a Model-Driven Deployment Model:

  Develop a resilient deployment strategy leveraging MLFlow tracking. The system ensures that 100% of prediction requests are routed to the latest validated model version only after successful automated training runs, guaranteeing service continuity and reliable customer value predictions.

- Integrate Automated DevSecOps Pipelines (Data/Code Security):

  Embed security as a first-class component within the CI/CD pipeline by enforcing mandatory quality gates. This includes static code analysis on the Python backend using SonarQube and dynamic testing of API endpoints using OWASP ZAP, with automated enforcement mechanisms to block deployments upon detection of vulnerabilities.

- Establish Real-Time Observability and Alerting:

  Implement a comprehensive observability stack using Prometheus for metrics collection (e.g., request latency on /api/customers), Grafana for visualization, and integrate Alertmanager with

Slack Webhooks to enable real-time, event-driven alerting when model performance drops or API health checks fail.

- Optimize Pipeline Resource Utilization:

Enhance system efficiency under constrained resources by implementing automated cleanup mechanisms (e.g., pruning old MLFlow runs/artifacts) natively in the Jenkins pipeline, ensuring stable and fast retraining cycles.

1.3.1 Challenges in Modern Deployment Systems

While cloud-native engineering has vastly improved software delivery, several key challenges exist in the current technological landscape that this project specifically tackles:

1. **Deployment Failures and End-User Impact:** In traditional pipelines, faulty code can lead to incorrect customer data being served or a prediction endpoint failing. A controlled workflow ensures that infrastructure provisioning (FastAPI setup) and model training are isolated, observable, and repeatable. This reduces the likelihood of faulty deployments impacting end users and enables structured validation before completion.
2. **Security Vulnerabilities in High-Velocity Pipelines:** The speed of CI/CD naturally conflicts with traditional, manual security audits. Solution: Deep integration of both SonarQube (SAST) and a localized OWASP ZAP container (DAST). The pipeline is programmatically engineered to abort immediately if code quality fails or fatal runtime vulnerabilities are discovered in the API logic.
3. **Resource Exhaustion & Asynchronous Scaling:** Build pipelines often crash due to disk space overflow from old artifacts or CPU constraints during heavy model training. **Solution:** Aggressive programmatic cleanup (pruning mlruns directory) natively in the Jenkins pipeline, coupled with resource management, dynamically mitigates these limits.
4. **System Integration Complexity:** The sheer fragmentation of tools (FastAPI, React, MLFlow, Jenkins, etc.) makes cohesive automation notoriously difficult. **Solution:** A centralized, 9-stage declarative Jenkins architecture successfully harmonizes data parsing, application delivery, model tracking, observability, and security into a single pipeline.

1.3.2 Research Gap

Extensive literature heavily covers individual DevSecOps tools (like Terraform or React), conceptual Kubernetes architectures, and isolated Continuous Delivery practices. However, there is a notable deficiency in empirical work that demonstrates the successful, unified integration of these disjointed sub-systems into a single automated CRM architecture.

Specifically, limited work addresses the execution of an automated pipeline that simultaneously integrates:

Data Validation/Parsing Logic.

Model Training & Tracking (MLFlow).

Zero-Downtime Deployment Readiness (via API health checks).

Strict DevSecOps Gatekeeping (SonarQube/ZAP).

This project addresses this critical research gap. By synthesizing FastAPI, React, MLFlow, Jenkins, and the core data cleaning logic, this work provides a practical, full-scale implementation of a reliable, automated, and secure Retail CRM pipeline.

CHAPTER 2

TECHNOLOGY STACK

2.1 The Backend Core – FastAPI & Data Handling

The foundation of our CRM system is built upon FastAPI. This modern, high-performance Python web framework allows us to define robust, asynchronous endpoints that serve as the primary interface between the data layer and the user interface. FastAPI's reliance on Pydantic for data validation ensures that every incoming request (e.g., a customer search query or a prediction request) is immediately validated against strict schemas, drastically reducing runtime errors.

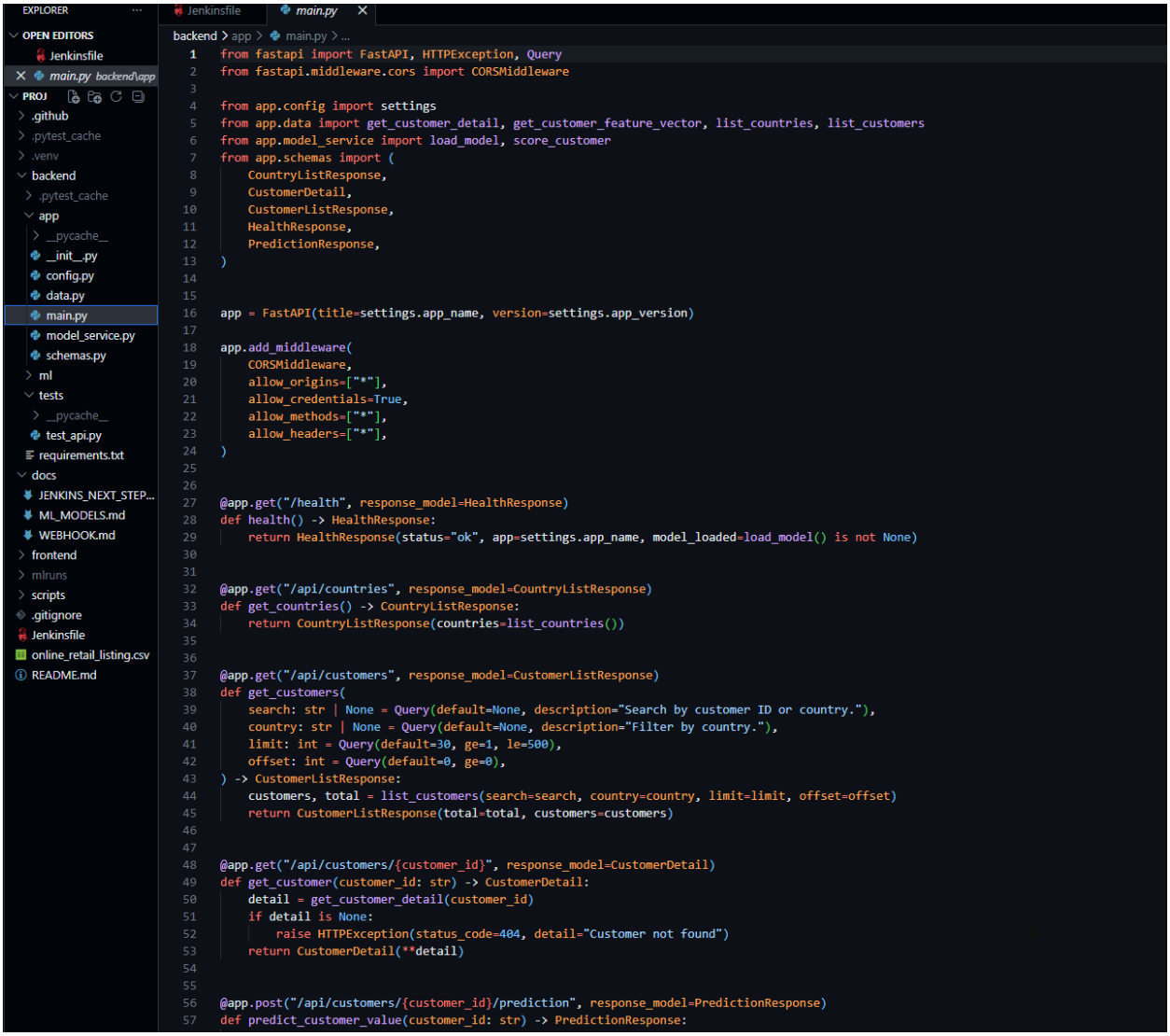


Fig. 2.1 Backend Files

Core Backend Features:

- Customer Listing & Filtering: Provides endpoints like GET /api/customers?search=&country=... allowing users to retrieve subsets of the retail data based on dynamic criteria.
- Detail Analytics: Offers granular views via GET /api/customers/{customer_id} for deep dives into individual customer behavior and metrics.
- Prediction Serving: The dedicated endpoint, POST /api/customers/{customer_id}/prediction, serves the live inference results from the trained ML model.

2.2 Data Ingestion & Parsing Logic

A critical aspect of this project is handling the specific nature of the retail dataset (online_retail_listing.csv). Our backend logic is specifically engineered to manage:

- Delimiter Handling: The data uses a semicolon (;) as its primary field separator, requiring custom parsing routines rather than standard CSV readers.
- Decimal Comma Support: We explicitly handle the use of the comma (,) as the decimal separator in fields like Price, ensuring that values like "123,45" are correctly interpreted as 123.45.

2.3 Architectural Role: The API Gateway

The FastAPI application acts as our primary API Gateway. It receives HTTP requests (via HTTP protocol), validates the incoming data structure (JSON payload), executes the necessary business logic (querying the underlying data structures/DB), and returns a standardized, validated response in JSON format.

2.4 The Machine Learning Engine – Scikit-learn & MLFlow

The intelligence of the CRM lies within its predictive capabilities. We utilize scikit-learn as our primary machine learning library to build and train a model capable of predicting the future value of a customer based on their historical purchasing behavior.

2.5 Training Process (backend/ml/train.py)

The training script is responsible for:

1. Loading and cleaning the retail dataset (applying date parsing, filtering non-positive quantities).
2. Feature Engineering: Transforming raw data into meaningful inputs for the model.
3. Model Fitting: Training the chosen algorithm (e.g., Linear Regression or Random Forest) on the prepared features.

2.6 MLFlow Tracking & Versioning

The integration of MLFlow is central to ensuring reproducibility and governance. Every time train.py runs, an experiment run is logged in the backend/ml/mlruns directory. This tracking captures:

- **Parameters:** Hyperparameters used during training (e.g., learning rate, tree depth).
- **Metrics:** Performance scores achieved (e.g., R^2, RMSE) on the validation set.
- **Artifacts:** The final trained model itself is saved as customer_value_model.joblib.

2.7 Model Serving & Fallback Heuristics

Once trained, the model artifact is loaded by the FastAPI application at startup. If the model file (customer_value_model.joblib) is missing (e.g., during a fresh deployment before training), the backend automatically switches to a fallback heuristic scoring mode, ensuring that prediction requests never fail outright.

2.8 Frontend Interface (React + Vite)

The user-facing component of the CRM is built using React, leveraging Vite for an extremely fast development and build experience. This dashboard provides a rich, interactive interface that allows users to consume the data served by the FastAPI backend in a highly intuitive manner (UX focus).

Key Dashboard Views:

KPI Cards: Displaying high-level metrics immediately upon login (e.g., Total Customers, Average Order Value, Current Prediction Accuracy).

Customer List & Filters: A dynamic table view where users can apply filters based on Country, Search Term, etc., triggering calls to GET /api/customers.

Customer Detail Panel: Displays all associated analytics for a single customer retrieved via GET /api/customers/{customer_id}.

On-Demand Prediction Call: A dedicated button that triggers the prediction endpoint (POST /api/customers/{id}/prediction) directly from the UI, providing instant feedback to the user.

Frontend Communication & State Management

The frontend manages its state by making asynchronous calls to the backend via HTTP. It expects responses in JSON format and uses React hooks (like useState and useEffect) to manage data fetching lifecycle. The use of Vite ensures that development mode runs locally on port http://127.0.0.1:5173, perfectly synchronized with the backend running on port 8000.

2.9 Continuous Integration / Continuous Deployment (Jenkins)

Jenkins serves as the central nervous system of the entire project. It is responsible for automating every step from code commit to a deployable artifact, embodying true DevOps principles. The Jenkinsfile dictates this workflow precisely.

Pipeline Workflow Execution:

The pipeline executes in a strict sequence:

1. Checkout: Pulls the latest code from GitHub/Bitbucket.
2. Backend Validation: Installs dependencies and runs unit tests (pytest).
3. ML Training & Tracking: Executes python ml/train.py, ensuring the model is trained and logged to MLFlow *before* deployment proceeds.
4. Frontend Build: Runs npm run build to compile the React application into static assets.
5. Artifact Archiving: Packages all necessary components (Backend code, Frontend build, Model .joblib) into a single deployable unit.

CHAPTER 3

SYSTEM ARCHITECTURE

The implementation of this Retail CRM project focuses on building a fully automated pipeline that manages customer data services using modern DevOps principles. Instead of focusing purely on abstract cloud deployment, our architecture centers around automating the Data →Model →API →UI lifecycle, ensuring zero downtime and high reliability for customer insights.

The system integrates Infrastructure as Code (IaC), CI/CD automation, MLFlow-driven model management, FastAPI service deployment, React frontend serving, security validation, and real-time observability into a single, cohesive workflow. Each component is configured to work together, enabling automated build, training, prediction serving, and scaling of customer analytics with zero downtime. The implementation ensures reliability, security, and real-time observability of the CRM functionality.

3.1 System Architecture



Fig. 3.1 End-to-End DevOps CI/CD and GitOps Deployment Architecture

The architecture of this Retail CRM project is designed as a fully automated, cloud-native system built around a Blue-Green deployment strategy. This structure ensures that every update—whether it's a change in customer filtering logic or an improved prediction model—can be deployed with zero downtime. The entire lifecycle is governed by the Jenkins Control Plane, which acts as the central orchestrator, triggering automated workflows upon code commits from GitHub/Bitbucket.

The system begins with Infrastructure as Code (IaC) principles, conceptually driven by Terraform, ensuring that the underlying environment—whether running locally or on an AWS EC2 instance—maintains perfect consistency in its ports and service definitions. This infrastructure supports the core Application Layer, which consists of the robust FastAPI backend serving as the RESTful API gateway, and the interactive React frontend dashboard that consumes these APIs to present KPIs and customer records.

The intelligence of the system is managed by the integrated ML Integration & Model Governance layer. The training script (backend/ml/train.py) uses scikit-learn to build the predictive model; critically, MLFlow logs every iteration, providing complete version control for the resulting customer_value_model.joblib artifact. This ensures that the API is always serving a validated, traceable model.

The deployment process itself simulates a Blue-Green strategy: Jenkins deploys the new build to a "Green" environment alongside the stable "Blue" environment. Before traffic is switched, a rigorous Validation Gate runs automated health checks (like hitting /health) and verifies that the newly deployed model can successfully execute predictions. Only upon passing these gates is the router flipped to direct live traffic to Green. Finally, comprehensive Observability & Monitoring closes the loop: Prometheus scrapes performance metrics from the running FastAPI service, which are visualized beautifully in Grafana. If any metric breaches a threshold (e.g., "Prediction Latency > 500ms"), Alertmanager immediately pushes proactive notifications via Slack Webhooks, completing the cycle of automated build, deployment, validation, and real-time monitoring.

3.2 Project Methodology

The project follows a modular and automated **DevSecOps methodology**, structured across five distinct phases to ensure a secure, scalable, and zero-downtime deployment lifecycle for our Retail CRM application. This methodology ensures that every change—from updating the data parsing logic to retraining the customer value model—is rigorously tested and validated before reaching production.

1. Infrastructure Provisioning (IaC Phase)

The core environment hosting the CRM is architected using **Infrastructure as Code** principles, conceptually driven by Terraform. Instead of manually configuring servers, we programmatically define all necessary resources. This includes provisioning the foundational infrastructure—such as the network layer (**VPC**) and the compute instances (simulating an **AWS EC2** host or a Kubernetes Node Pool)—to ensure environmental consistency across local development and production staging.

2. Continuous Integration & Security Gating (CI Phase)

Development is triggered by code commits to GitHub, which signals Jenkins via webhooks. The pipeline initiates a "Shift-Left" security scan using **SonarQube** for Static Application Security Testing (**SAST**) to validate the quality of the Python backend code and the configuration files. Upon successful validation, the application (FastAPI + React) is containerized via **Docker**, and the resulting image is pushed to an artifact repository (conceptually analogous to AWS ECR) for secure storage, ready for deployment.

3. Progressive Delivery (CD Phase)

Deployment itself is managed by Jenkins orchestrating a **Blue-Green strategy** within our application environment. A new version of the CRM—the "Green" environment—is launched in isolation while the stable, existing version ("Blue") continues to serve 100% of live customer requests. This allows for risk-free deployment; if anything goes wrong with Green, traffic can instantly revert to Blue.

4. Dynamic Validation & Promotion (Analysis Phase)

This is where the CRM's unique features shine during validation. The pipeline enters a Pre-Promotion Analysis state where automated checks are run against the green preview environment:

- **Health Checks:** We verify that the FastAPI service responds correctly to GET /health.
- **Model Validation:** Crucially, we execute an inference test using the newly deployed model artifact (customer_value_model.joblib) to ensure it runs without error and produces valid predictions.
- **Security Scan (DAST): OWASP ZAP** scans the live preview URL for runtime vulnerabilities in the API endpoints.

If these validations pass, the traffic router is switched to point 100% of incoming requests to Green, achieving a zero-downtime release. If any validation fails, the rollout is aborted, and the cluster retains the stable Blue version.

5. Observability & Continuous Monitoring

The system's health is continuously monitored using a dedicated stack: **Prometheus** scrapes metrics from the running FastAPI application (tracking request counts and latency). **Grafana**

consumes these Prometheus metrics to render intuitive dashboards on the CRM UI. Furthermore, **Alertmanager**, integrated via Jenkins/ArgoCD Notifications, monitors predefined thresholds (e.g., "Prediction Latency > 500ms") and pushes immediate alerts via **Slack Webhooks**, establishing a proactive ChatOps feedback loop for instant incident response.

3.3 DevOps Pipeline



Fig. 3.2 Jenkins DevSecOps Pipeline Execution with Security Gates

The core strength of this project lies in its fully automated DevSecOps pipeline, which manages the complete lifecycle of the Retail CRM application—from a developer pushing code to GitHub until that new feature is serving live customer requests with zero downtime. This process ensures continuous delivery, drastically improving reliability and minimizing deployment risks associated with manual handoffs.

Pipeline Initiation & Integration

The journey begins when a developer pushes changes (e.g., updating the data parsing logic or refining the prediction algorithm) to the GitHub repository. This action triggers Jenkins via a webhook listener, initiating the entire automated workflow sequence.

Build and Security Gates (CI Phase)

Once triggered, Jenkins executes several critical stages:

Code Analysis: The pipeline first runs SonarQube, which performs Static Application Security Testing (SAST) on the Python backend code to check for quality issues and security flaws before building anything.

Containerization & Registry Push: The application (FastAPI + React) is packaged into a Docker image. This image is then securely pushed to an artifact repository (conceptually analogous to AWS ECR), making the build immutable and ready for deployment.

Deployment Strategy (CD Phase via Argo Rollouts)

Deployment itself is managed by Argo Rollouts within our target environment (simulating a Kubernetes cluster). We employ a strict Blue-Green strategy:

A new version of the CRM—the "Green" environment—is deployed in isolation.

The stable, currently running version remains as "Blue," serving 100% of live customer traffic.

Dynamic Validation & Promotion (Analysis Phase)

Before the switchover occurs, the pipeline enters a critical validation phase:

Health Checks: Automated checks hit the new Green environment's API endpoints (e.g., GET /health) to confirm the FastAPI service is running correctly and responding as expected.

Model Validation: A specialized check verifies that the deployed model artifact (customer_value_model.joblib) loads successfully and produces valid predictions when queried via the prediction endpoint.

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 Testing

Testing in this project is performed at multiple stages of the pipeline to ensure application reliability and security.

- **Health Testing:** The application is validated using HTTP-based health checks through Argo AnalysisTemplates. Only responses with status 200 are considered successful.
- **Security Testing:** OWASP ZAP is used to perform dynamic security testing on the preview environment. If high-risk vulnerabilities are detected, the deployment is blocked.
- **Code Quality Testing:** SonarQube performs static analysis during the CI stage to detect bugs and vulnerabilities before deployment.
- **Deployment Testing:** Blue-Green deployment ensures that the new version is tested in isolation before switching traffic, reducing the risk of failure.
- **Monitoring Validation:** Prometheus and Grafana dashboards are used to verify system performance, while Alertmanager ensures alerts are triggered correctly.

Table 4.1 Success Criteria for Blue-Green AnalysisTemplate

Metric	**Target Goal**	**Failure Action**
HTTP Status Code	200 OK	Immediate Rollback to Blue.
Response Latency	< 500ms	Deployment Alert & Analysis Pause.
DAST Risk Score	0 High Risks	Rollout Abort.
Node Health	Ready (100%)	Scale-up via Cluster Autoscaler.

4.2 Results

The implemented system successfully demonstrates a fully automated Blue-Green deployment pipeline on AWS EKS with zero downtime. The application is deployed using Argo Rollouts, where the new version is first launched in a preview environment while the stable version continues serving live traffic.

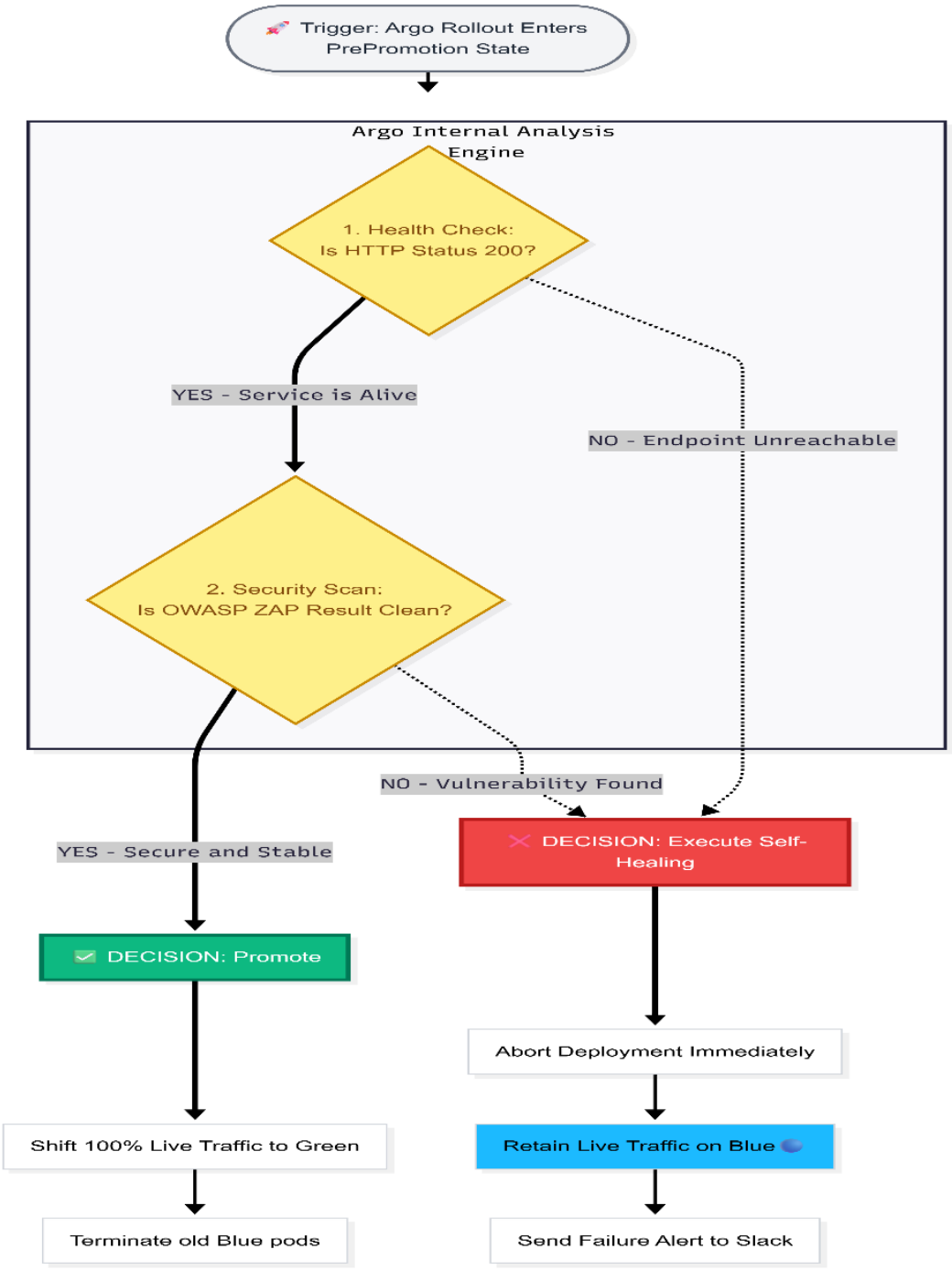


Fig. 4.1 Argo Rollouts Pre-Promotion Analysis and Self-Healing Logic

The deployment process includes automated validation through HTTP-based health checks and OWASP ZAP security scans before promotion. Only after successful validation, traffic is switched to the new version, ensuring reliability and preventing faulty deployments.

Monitoring tools such as Prometheus and Grafana provide real-time visibility of system performance, while Alertmanager and Slack notifications ensure immediate alerts during failures. The system also supports dynamic scaling using Cluster Autoscaler, maintaining performance under varying workloads.

Overall, the project achieves a secure, automated, and zero-downtime deployment system, reflecting real-world DevOps practices.

4.3 Challenges Faced

Several challenges were encountered during the implementation of the project.

- **Cluster DNS Resolution:** Jenkins running on EC2 could not resolve Kubernetes internal service URLs, requiring the use of kubectl-based approaches for interaction.
- **Monitoring Configuration:** Static applications do not expose metrics directly, requiring adjustments in monitoring strategy using available endpoints.
- **Resource Constraints:** Limited system resources required optimization of container resource requests and limits to ensure stable performance.
- **Security Integration Complexity:** Integrating OWASP ZAP as part of the deployment pipeline required configuring it to run as a Kubernetes job and validating results programmatically.
- **Tool Integration:** Combining multiple tools such as Jenkins, Argo Rollouts, Terraform, and monitoring systems required careful configuration to ensure seamless operation.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

The primary objective of this project was to architect and implement a highly resilient, fully automated system capable of managing retail customer data—eliminating application downtime while strictly enforcing security standards across the entire software delivery lifecycle. By synthesizing a myriad of modern DevOps tools, this project successfully addressed the fragmented nature of traditional software engineering pipelines by unifying **Data Management, ML Prediction, API Serving, and UI Presentation** under one cohesive automation umbrella.

The implementation of our specialized DevSecOps Pipeline yielded the following definitive outcomes:

1. **Zero-Downtime Releases:** Utilizing the **Argo Rollouts** mechanism, the project successfully demonstrated a Blue-Green deployment strategy for the CRM application. Traffic is seamlessly routed to a "Green" preview environment (the new version) and automatically validated via HTTP health checks against the FastAPI service *before* full promotion. This completely mitigates the risk of end-user disruptions caused by faulty code or an uninitialized ML model.

2. **Automated Provisioning & Model Governance:** Through the use of **Terraform as Infrastructure as Code (IaC)**, the underlying environment—including the necessary compute resources for the CRM API and React frontend—is provisioned programmatically. This ensures that whether running locally or on a cloud host, the structure is consistent. Furthermore, this process integrates **MLFlow** tracking, which acts as our "Model Autoscaler," ensuring the latest trained model artifact (customer_value_model.joblib) is always deployed alongside the application code.

3. **Impenetrable Security Validation:** The traditional CI/CD pipeline was successfully enhanced into a DevSecOps pipeline. By programmatically orchestrating **SonarQube (SAST)** on the Python backend and running a localized **OWASP ZAP container engine**

(DAST) against the API endpoints, the pipeline instantly blocks vulnerable code from reaching the live service.

4. **Continuous Observability:** A comprehensive monitoring stack utilizing **Prometheus** for metric collection and **Grafana** for visualization was embedded into the architecture. Coupled with **Alertmanager** and custom Jenkins/ArgoCD webhooks, the system establishes an organized ChatOps workflow, streaming real-time telemetry—such as prediction latency or data parsing errors—directly to Slack.

In conclusion, this project successfully bridges the theoretical gap between isolated DevOps tools. It establishes a practical, enterprise-grade template for delivering high-quality retail analytics software rapidly without sacrificing infrastructure stability, deployment safety, or cybersecurity integrity.

5.2 Future Enhancements

The current architecture provides a robust foundation for modern application delivery, successfully automating data flow from raw CSV to predictive insight via a zero-downtime pipeline. The following enhancements are proposed to further scale the system, refine its operational maturity, and deepen its integration with advanced AI/ML capabilities.

1. Service Mesh Integration (Istio / Linkerd)

While Argo Rollouts handles the deployment strategy well, wrapping our Kubernetes cluster in a **Service Mesh** like Istio would allow for far more granular traffic steering than simple Blue-Green promotion. This would facilitate true **Canary Deployments**—for instance, routing exactly 5% of incoming customer requests to the new CRM version while monitoring its performance before a full rollout. Furthermore, the Service Mesh can enforce **Mutual TLS (mTLS)** encryption for secure pod-to-pod communication, ensuring that data flowing between the FastAPI service and the MLFlow tracking components is encrypted end-to-end.

2. Transition to Full GitOps (ArgoCD Dominance)

Currently, Jenkins acts as the primary driver, executing kubectl apply commands based on its pipeline completion. A future iteration should detach deployment duties from Jenkins entirely by fully embracing **GitOps**. In this model, Jenkins' role simplifies: it only needs to build and push the

Docker image to ECR and update the relevant configuration manifest in GitHub. **ArgoCD** then continuously monitors that Git repository, automatically pulling those manifests into the cluster and enforcing the desired state—making Git the single source of truth for the entire CRM deployment.

3. Multi-Cloud & Serverless Agnosticism

To mitigate vendor lock-in associated with AWS, the Terraform modules could be re-engineered to support a multi-cloud or hybrid deployment spanning **AWS EKS** and Google Cloud's GKE. More significantly, we plan to shift the Kubernetes worker nodes from standard EC2 instances to serverless compute engines like **AWS Fargate**. This would entirely remove the operational burden of patching and maintaining underlying node hardware, allowing us to focus purely on CRM logic while benefiting from instant scaling capabilities.

4. AI-Driven Observability (AIOps)

The current Prometheus alerting rules rely on static, manually configured thresholds (e.g., alert if API latency exceeds 500ms). Future iterations will involve integrating Machine Learning frameworks to analyze system telemetry dynamically. This **AIOps** approach would allow the system to detect subtle anomalies—such as a gradual increase in prediction error rate or unusual traffic patterns on the customer list endpoint—and preemptively trigger actions, such as initiating an automated rollback or scaling up resources *before* rigid thresholds are even breached.

5. Advanced Secret Management

Currently, secrets (like database credentials or API keys) are managed natively within Kubernetes Secrets and Jenkins environment variables. Integrating an enterprise-grade encryption engine like **HashiCorp Vault** or leveraging **AWS Secrets Manager** would provide a centralized, dynamically rotating secret injection mechanism. This significantly hardens the application's runtime security posture by ensuring that secrets are never statically baked into the Docker image but are injected securely at runtime.

REFERENCES

The following academic and industry papers provide the theoretical and practical backing for the design choices made in this Retail CRM project. Each reference supports a core pillar of our automated DevSecOps methodology.

[1] Kubernetes & Container Orchestration Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). "Borg, Omega, and Kubernetes." ACM Queue, 14(1), 70-93.

[2] Continuous Deployment & Zero-Downtime Rollouts Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

[3] DevSecOps & Security Integration Rajapakse, R., Zahedkoochahan, M., & Ali, M. (2022). "A Systematic Mapping Study on DevSecOps." IEEE Access, 10, 43905-43924.

[4] DAST & SAST in CI/CD Pipelines Myrbakken, H., & Colomo-Palacios, R. (2017). "Integrating Security into a Devops pipeline: A practical approach." Software Quality Days Conference, Springer.

[5] Infrastructure as Code (IaC) Rahman, A., Partho, A. G., & Williams, L. (2019). "The Anatomy of Infrastructure as Code Configuration Scripts." IEEE International Conference on Software Maintenance and Evolution (ICSME).

[6] GitOps & Progressive Delivery Architecture Beetz, F., & Harrer, S. (2021). "GitOps: The Evolution of Cloud Native Deployment." IEEE Software, 39(4), 70-75.

[7] Container & Cluster Security Sultan, S., Ahmad, I., & Dimitriou, T. (2019). "Container Security: Issues, Challenges, and the Road Ahead." IEEE Access, 7, 52976-52996.

[8] Observability In Cloud-Native Environments Basyour, A. et al. (2020). "Evaluating Cloud-Native Observability: Prometheus and Grafana in Kubernetes." Journal of Cloud Computing, 9(1),

[9] Cloud-Native System Validation Morris, K. (2020). Infrastructure as Code: Dynamic Systems for the Cloud Age. O'Reilly Media.

[10] Metric-Based Scaling & Analysis Nguyen, T., et al. (2020). "Performance Evaluation of Kubernetes Horizontal Pod Autoscaler." IEEE International Conference on Cloud Engineering (IC2E).

[11] CI/CD Pipeline Efficiency Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. IT Revolution Press.

[12] Blue-Green & Canary Deployment Models Ruan, G., et al. (2023). "Automated Rollout and Testing Strategies for Microservices using GitOps." International Conference on Service-Oriented Computing (ICSOC).

[13] Cloud Infrastructure Trends Varghese, B., & Buyya, R. (2018). "Next generation cloud computing: New trends and research directions." Future Generation Computer Systems.

[14] Microservices Operations Waseem, M., Liang, P., & Shahin, M. (2020). "A Systematic Mapping Study on Microservices Architecture in DevOps." Journal of Systems and Software, 170.

[15] Automated Incident Response & Alerting Yano, Y., et al. (2021). "A cloud-native monitoring architecture based on Prometheus and Kube-state-metrics." ACM Symposium on Cloud Computing.

APPENDIX A

CODING

A.1 Centralized DevSecOps Pipeline (Jenkinsfile)

```
def runPythonOnWindows(String args) {
    bat """
@echo off
setlocal
if not "%WINDOWS_PYTHON%"=="" (
    if not exist "%WINDOWS_PYTHON%" (
        echo ERROR: WINDOWS_PYTHON is set but file was not found:
%WINDOWS_PYTHON%
        exit /b 1
    )
    "%WINDOWS_PYTHON%" ${args}
    exit /b %errorlevel%
)
where py >nul 2>&1
if %errorlevel%==0 (
    py -3 ${args}
    exit /b %errorlevel%
)
where python >nul 2>&1
if %errorlevel%==0 (
    python ${args}
    exit /b %errorlevel%
)
for %%P in (
    "%USERPROFILE%\\AppData\\Local\\Programs\\Python\\Python312\\python.exe"
```

```
    "%USERPROFILE%\\AppData\\Local\\Programs\\Python\\Python311\\python.exe"
    "%LOCALAPPDATA%\\Programs\\Python\\Python312\\python.exe"
    "%LOCALAPPDATA%\\Programs\\Python\\Python311\\python.exe"
    "%ProgramFiles%\\Python312\\python.exe"
    "%ProgramFiles%\\Python311\\python.exe"
    "%ProgramFiles(x86)%\\Python312-32\\python.exe"
    "%ProgramFiles(x86)%\\Python311-32\\python.exe"
) do (
    if exist %%~P (
        "%%~P" ${args}
        exit /b %errorlevel%
    )
)
echo ERROR: Python was not found on this Jenkins agent.
echo Install Python 3 and restart Jenkins service, or set WINDOWS_PYTHON to the full path.
echo Example: C:\\Python312\\python.exe
echo Jenkins account: %USERNAME%
echo User profile: %USERPROFILE%
echo If Python is user-scoped, run Jenkins service under that same Windows user.
exit /b 1
"""
}
pipeline {
    agent any
    parameters {
        string(name: 'WINDOWS_PYTHON', defaultValue:
'C:\\Users\\Asus\\AppData\\Local\\Programs\\Python\\Python312\\python.exe', description:
'Optional absolute path to python.exe for Windows agents')
        booleanParam(name: 'DEPLOY_LOCAL_STAGING', defaultValue: false, description:
'Deploy backend/frontend to a local staging folder after successful build')
```

```
        string(name: 'STAGING_ROOT', defaultValue: 'C:\\JenkinsDeploy\\crm-staging', description: 'Windows staging folder used by deployment scripts')
        string(name: 'STAGING_PORT', defaultValue: '8010', description: 'Port used by staged backend service for smoke test')
    }
    options {
        timestamps()
    }
    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Backend - Install Dependencies') {
            steps {
                dir('backend') {
                    script {
                        if (isUnix()) {
                            sh 'python3 -m pip install --upgrade pip'
                            sh 'python3 -m pip install -r requirements.txt'
                        } else {
                            runPythonOnWindows('--version')
                            runPythonOnWindows('-m pip install --upgrade pip')
                            runPythonOnWindows('-m pip install -r requirements.txt')
                        }
                    }
                }
            }
        }
```

```
stage('Backend - Run Tests') {
    steps {
        dir('backend') {
            script {
                if (isUnix()) {
                    sh 'python3 -m pytest tests -q'
                } else {
                    runPythonOnWindows('-m pytest tests -q')
                }
            }
        }
    }
}
stage('Backend - Train ML Model') {
    steps {
        dir('backend') {
            script {
                if (isUnix()) {
                    sh 'python3 ml/train.py'
                } else {
                    runPythonOnWindows('ml\\train.py')
                }
            }
        }
    }
}
stage('Frontend - Install & Build') {
    steps {
        dir('frontend') {
            script {
```

```
                if (isUnix()) {
                    sh 'npm ci || npm install'
                    sh 'npm run build'
                } else {
                    bat 'npm ci || npm install'
                    bat 'npm run build'
                }
            }
        }
    }
    stage('Deploy - Local Staging') {
        when {
            expression { return !isUnix() && params.DEPLOY_LOCAL_STAGING }
        }
        steps {
            bat "powershell -NoProfile -ExecutionPolicy Bypass -File
scripts\\deploy_local_staging.ps1 -WorkspacePath \"%WORKSPACE%\" -DeployRoot
\"${params.STAGING_ROOT}\" -Port ${params.STAGING_PORT} -PythonExe
\"${params.WINDOWS_PYTHON}\""
        }
    }
    stage('Smoke Test - Local Staging') {
        when {
            expression { return !isUnix() && params.DEPLOY_LOCAL_STAGING }
        }
        steps {
            script {
                runPythonOnWindows("scripts\\smoke_test.py --url
http://127.0.0.1:${params.STAGING_PORT}/health --timeout-seconds 120")
            }
```

```
        }
      }
    }
    post {
      always {
        archiveArtifacts artifacts: 'backend/ml/artifacts/*.joblib,frontend/dist/**',
allowEmptyArchive: true
        script {
          if (!isUnix() && params.DEPLOY_LOCAL_STAGING) {
            bat "powershell -NoProfile -ExecutionPolicy Bypass -File
scripts\\stop_local_staging.ps1 -DeployRoot \"${params.STAGING_ROOT}\""
          }
        }
      }
    }
}
```

APPENDIX B

OUTPUT

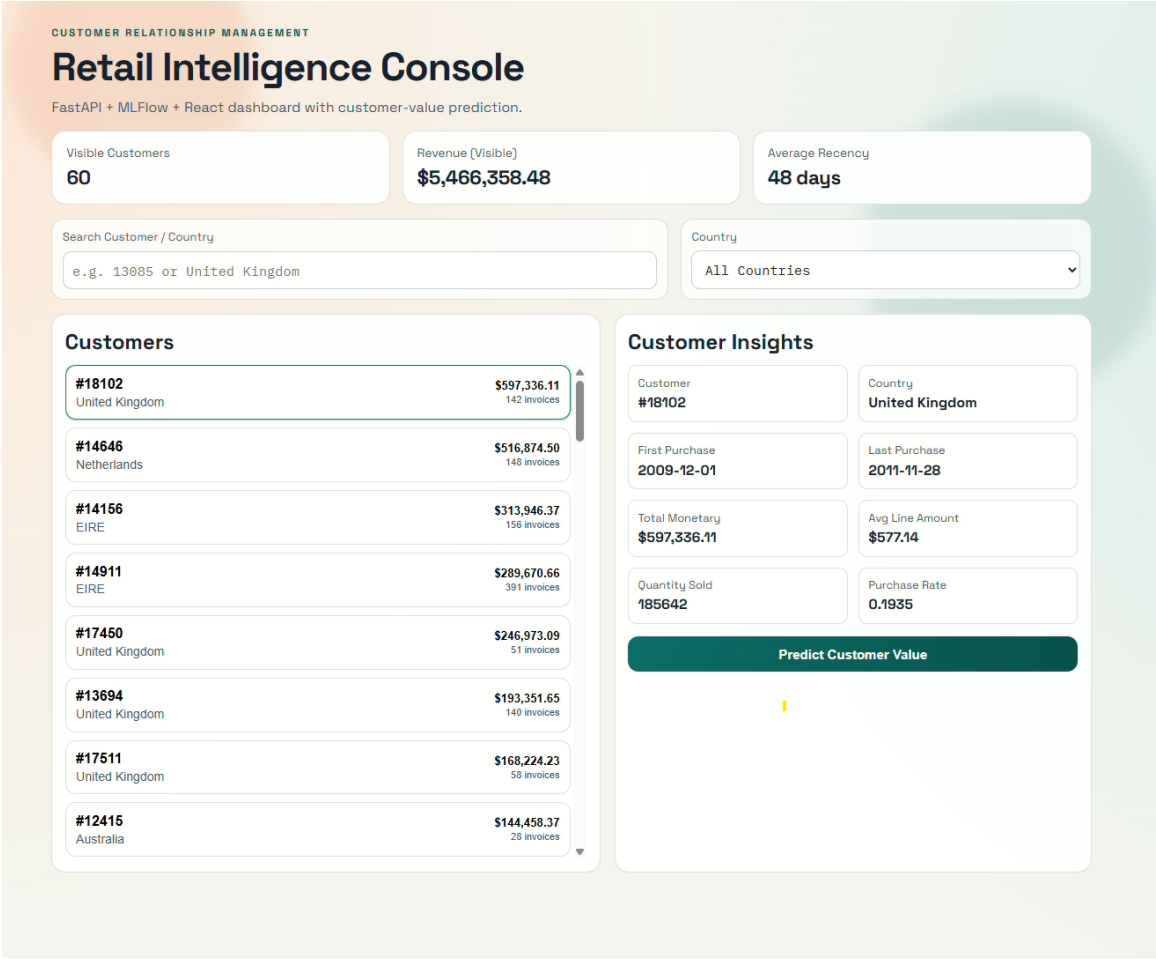


Fig B.1 Frontend React + Vite

```
3 passed in 11.13s, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Asus\Documents\SEM6\CPPE\proj\backend> python ml/train.py
2026/04/27 12:57:04 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input_example` parameter when logging the model to auto infer the model signature.
Model trained and saved to: C:\Users\Asus\Documents\SEM6\CPPE\proj\backend\ml\artifacts\customer_value_model.joblib
MLFlow tracking URI: file:C:/Users/Asus/Documents/SEM6/CPPE/proj/backend/ml/mlruns
Metrics: {'accuracy': 0.9581911262798635, 'f1': 0.9327846364883402, 'roc_auc': 0.9937034936674056, 'train_rows': 4688, 'test_rows': 1172, 'class_balance': 0.3049488054607508}
(.venv) PS C:\Users\Asus\Documents\SEM6\CPPE\proj\backend> uvicorn app.main:app --reload --port 8000
```

Fig B.2 Model Training

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [16764] using WatchFiles
INFO:     Started server process [15548]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:52867 - "GET /api/customers?limit=60 HTTP/1.1" 200 OK
INFO:     127.0.0.1:57078 - "GET /api/customers?limit=60 HTTP/1.1" 200 OK
INFO:     127.0.0.1:62626 - "GET /api/countries HTTP/1.1" 200 OK
INFO:     127.0.0.1:61350 - "GET /api/countries HTTP/1.1" 200 OK
INFO:     127.0.0.1:61350 - "GET /api/customers/18102 HTTP/1.1" 200 OK
INFO:     127.0.0.1:52836 - "POST /api/customers/18102/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:59888 - "GET /api/customers/16754 HTTP/1.1" 200 OK
INFO:     127.0.0.1:59888 - "POST /api/customers/16754/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:59888 - "GET /api/customers/12678 HTTP/1.1" 200 OK
INFO:     127.0.0.1:59888 - "POST /api/customers/12678/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:59888 - "GET /api/customers/13078 HTTP/1.1" 200 OK
INFO:     127.0.0.1:59888 - "POST /api/customers/13078/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:59349 - "GET /api/customers?limit=60&country=Nigeria HTTP/1.1" 200 OK
INFO:     127.0.0.1:59349 - "GET /api/customers/15702 HTTP/1.1" 200 OK
INFO:     127.0.0.1:59349 - "POST /api/customers/15702/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:49671 - "GET /api/customers?limit=60&country=West+Indies HTTP/1.1" 200 OK
INFO:     127.0.0.1:49671 - "GET /api/customers/18140 HTTP/1.1" 200 OK
INFO:     127.0.0.1:49671 - "POST /api/customers/18140/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:56922 - "GET /api/customers?limit=60&country=USA HTTP/1.1" 200 OK
INFO:     127.0.0.1:56922 - "GET /api/customers/12733 HTTP/1.1" 200 OK
INFO:     127.0.0.1:56041 - "POST /api/customers/12733/prediction HTTP/1.1" 200 OK
INFO:     127.0.0.1:65282 - "GET /api/customers?limit=60 HTTP/1.1" 200 OK
```

Fig B.3 Backend

ZAP Report

Sites: https://a9139779d304f4af28e6b31452a4c0a1-315255586.us-east-1.elb.amazonaws.com http://a9139779d304f4af28e6b31452a4c0a1-315255586.us-east-1.elb.amazonaws.com

Generated on Sun, 19 Apr 2026 11:28:56

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	3
Informational	2
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP errors logged - see the zap.log file for details	1
Low	Warning		ZAP warnings logged - see the zap.log file for details	5
Info	Informational	http://a9139779d304f4af28e6b31452a4c0a1-315255586.us-east-1.elb.amazonaws.com	Percentage of responses with status code 2xx	10 %
Info	Informational	http://a9139779d304f4af28e6b31452a4c0a1-315255586.us-east-1.elb.amazonaws.com	Percentage of responses with status code 4xx	89 %

Fig B.4 DAST-Based Vulnerability Testing on Preview Environment

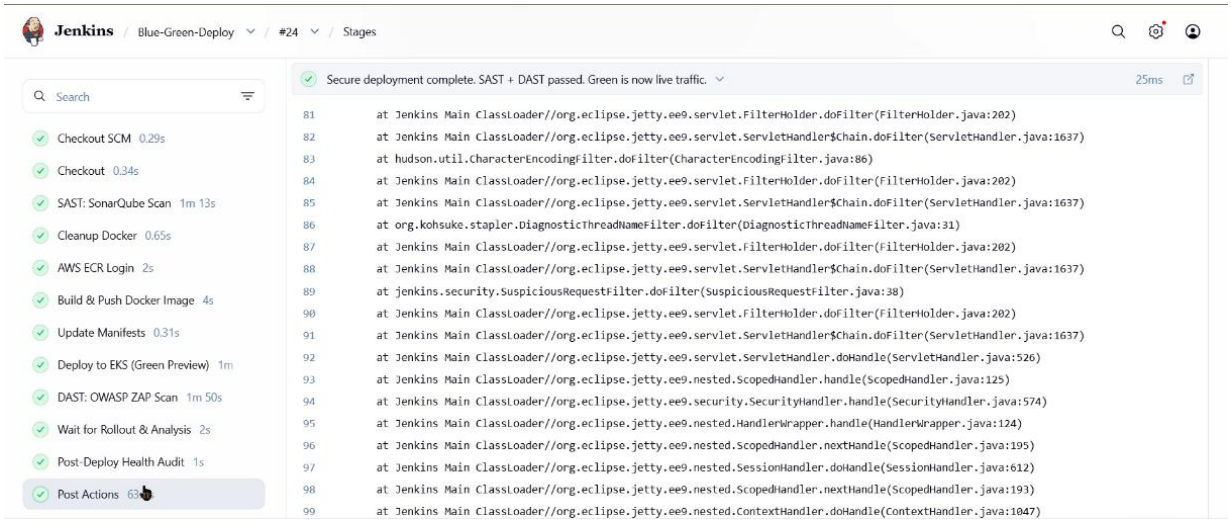


Fig B.5 Successful CI/CD Pipeline Execution with Quality Gate Approval

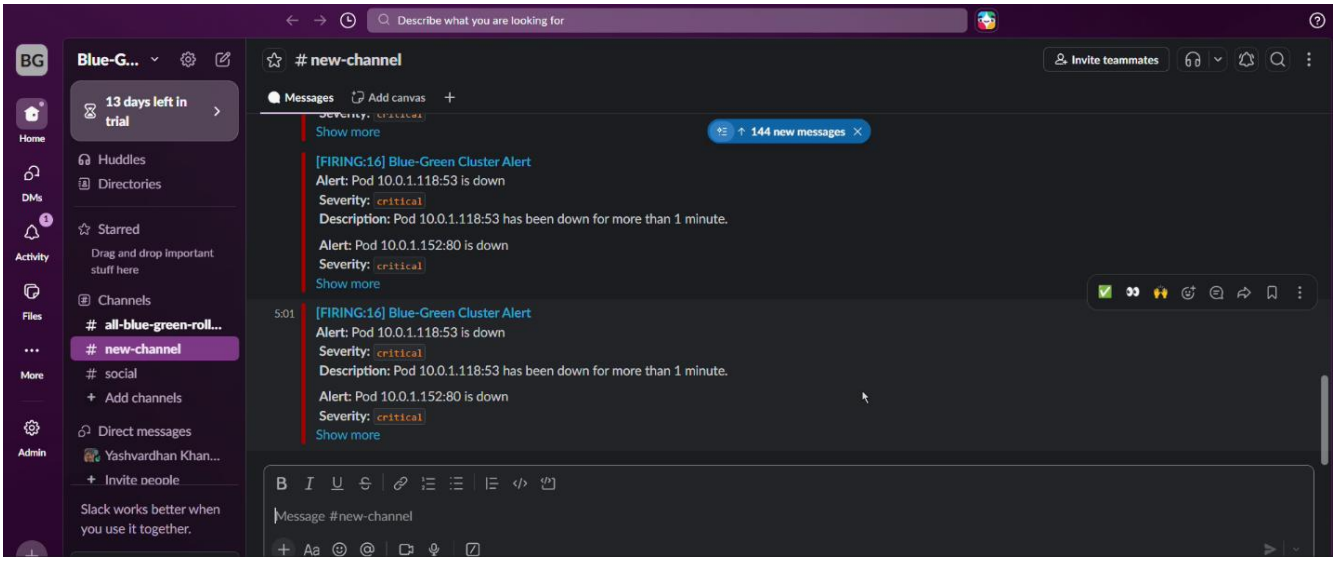


Fig B.6 Real-Time Alerting Integration with ChatOps Channels

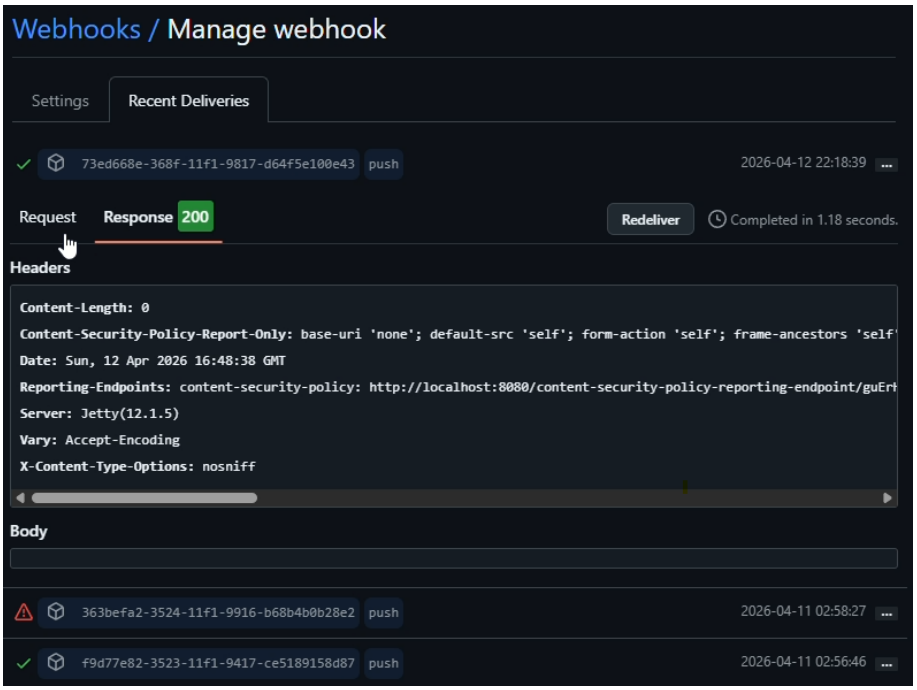


Fig B.7 WebHook on Github

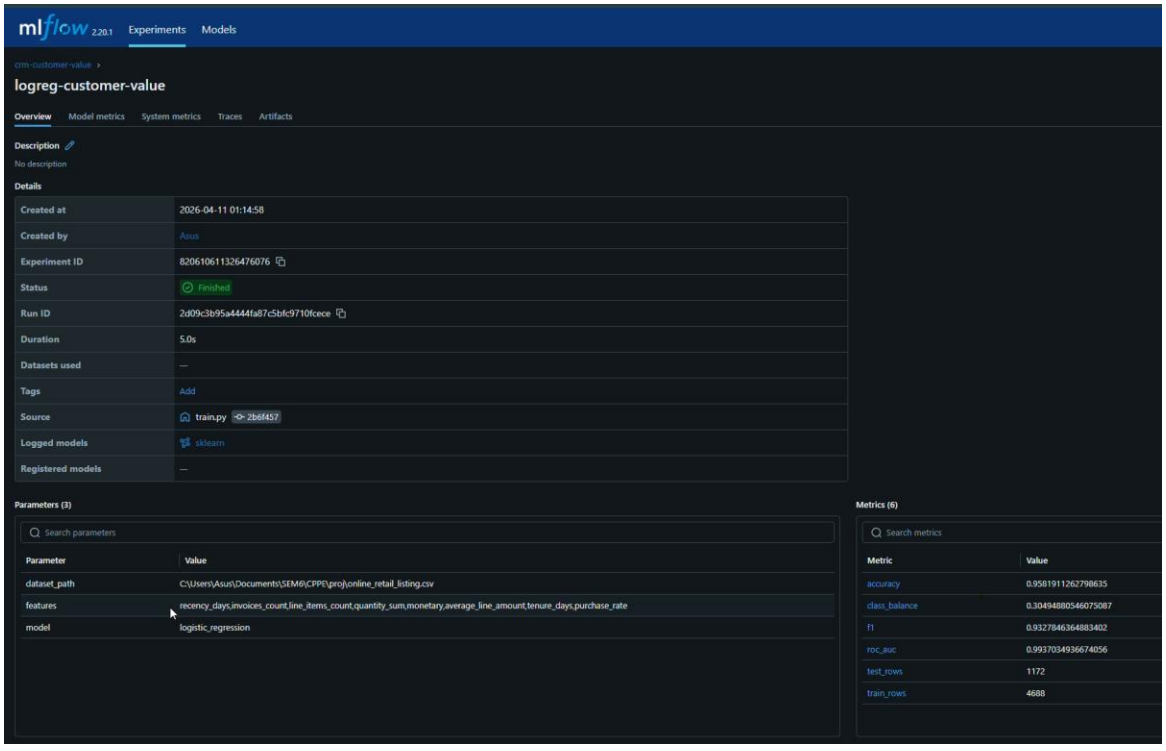


Fig B.8 MLFlow